

GroverOperator

```
class QiskitCircuitLibraryGroverOperator(oracle, state_preparation=None,
    zero_reflection=None, reflection_qubits=None, insert_barriers=False, mcx_mode='noancilla',
    name='Q')
```

GitHub

Base: QuantumCircuit

The Grover operator.

Grover's search algorithm [1, 2] consists of repeated applications of the so-called Grover operator used to amplify the amplitudes of the desired output states. This operator, Q , consists of the phase oracle, S_f , zero phase-shift or zero reflection, S_0 , and an input state preparation A :

$$Q = AS_0A^\dagger S_f$$

In the standard Grover search we have $A = H^{\otimes n}$:

$$Q = H^{\otimes n}S_0H^{\otimes n}S_f = DS_f$$

The operation $D = H^{\otimes n}S_0H^{\otimes n}$ is also referred to as diffusion operator. In this formulation we can see that Grover's operator consists of two steps: first, the phase oracle multiplies the good states by -1 (with S_f) and then the whole state is reflected around the mean (with D).

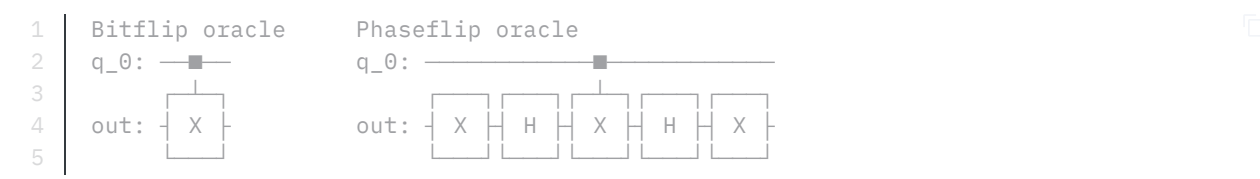
This class allows setting a different state preparation, as in quantum amplitude amplification (a generalization of Grover's algorithm), A might not be a layer of Hadamard gates [3].

The action of the phase oracle S_f is defined as

$$S_f : |x\rangle \mapsto (-1)^{f(x)}|x\rangle$$

where $f(x) = 1$ if x is a good state and 0 otherwise. To highlight the fact that this oracle flips the phase of the good states and does not flip the state of a result qubit, we call S_f a phase oracle.

Note that you can easily construct a phase oracle from a bitflip oracle by sandwiching the controlled X gate on the result qubit by a X and H gate. For instance



There is some flexibility in defining the oracle and A operator. Before the Grover operator is applied in Grover's algorithm, the qubits are first prepared with one application of the A operator (or Hadamard gates in the standard formulation). Thus, we always have operation of the form $AS_fA^\dagger$. Therefore it is possible to move bitflip logic into A and leaving the oracle only to do phaseflips via Z gates based on the bitflips. One possible use-case for this are oracles that do not uncompute the state qubits.

The zero reflection S_0 is usually defined as

$$S_0 = 2|0\rangle\langle 0|^{\otimes n}|0\rangle\langle 0|^{\otimes n} - I_n$$

where I_n is the identity on n qubits. By default, this class implements the negative version $2|0\rangle\langle 0|^{\otimes n}|0\rangle\langle 0|^{\otimes n} - I_n$, since this can simply be implemented with a multi-controlled Z sandwiched by X gates on the target qubit and the introduced global phase does not matter for Grover's algorithm.

Examples

```
1 >>> from qiskit.circuit import QuantumCircuit
2 >>> from qiskit.circuit.library import GroverOperator
3 >>> oracle = QuantumCircuit(2)
4 >>> oracle.z(0) # good state = first qubit is |1>
5 >>> grover_op = GroverOperator(oracle, insert_barriers=True)
6 >>> grover_op.decompose().draw()
```

```
1 >>> oracle = QuantumCircuit(1)
2 >>> oracle.z(0) # the qubit state |1> is the good state
3 >>> state_preparation = QuantumCircuit(1)
4 >>> state_preparation.zy(0.2, 0) # non-uniform state preparation
5 >>> grover_op = GroverOperator(oracle, state_preparation)
6 >>> grover_op.decompose().draw()
```

```
1 >>> oracle = QuantumCircuit(4)
2 >>> oracle.z(3)
3 >>> reflection_qubits = [0, 3]
4 >>> state_preparation = QuantumCircuit(4)
5 >>> state_preparation.cry(0.1, 0, 3)
6 >>> state_preparation.zy(0.5, 3)
7 >>> grover_op = GroverOperator(oracle, state_preparation,
8 ... reflection_qubits=reflection_qubits)
9 >>> grover_op.decompose().draw()
```

```
1 >>> mark_state = Statevector.from_label('011')
2 >>> diffuse_operator = 2 * DensityMatrix.from_label('000') - Operator.from_label('111')
3 >>> grover_op = GroverOperator(oracle=mark_state, zero_reflection=diffuse_operator)
4 >>> grover_op.decompose().draw(fold=70)
```

See also

The `grover_operator` implements the same functionality but keeping the `RCXGate` abstract, such that the compiler may choose the optimal decomposition. We recommend using `grover_operator` for performance reasons, which does not wrap the circuit into an opaque gate.

References

[1]: L. K. Grover (1996), A fast quantum mechanical algorithm for database search,

arXiv:quant-ph/9605043

[2]: I. Chuang & M. Nielsen, Quantum Computation and Quantum Information,

Cambridge: Cambridge University Press, 2000, Chapter 6.1.2.

[3]: Brassard, G., Hoyer, P., Mosca, M., & Tapp, A. (2000).

Quantum Amplitude Amplification and Estimation. arXiv:quant-ph/0005055

Deprecated since version 1.3_pending

The class `qiskit.circuit.library.grover_operator.GroverOperator` is pending deprecation as of qiskit 1.3. It will be marked deprecated in a future release, and then removed no earlier than 3 months after the release date. Use `qiskit.circuit.library.grover_operator` instead.

Parameters

- `oracle` (`Union[QuantumCircuit, Statevector]`) – The phase oracle implementing a reflection about the bad state. Note that this is not a bitflip oracle, see the docstring for more information.
- `state_preparation` (`Optional[QuantumCircuit]`) – The operator preparing the good and bad state. For Grover's algorithm, this is a n -qubit Hadamard gate and for amplitude amplification or estimation the operator A .
- `zero_reflection` (`Optional[Union[QuantumCircuit, DensityMatrix, Operator]]`) – The reflection about the zero state, S_0 .
- `reflection_qubits` (`Optional[List[int]]`) – Qubits on which the zero reflection acts on.
- `insert_barriers` (`bool`) – Whether barriers should be inserted between the reflections and A .
- `mcx_mode` (`str`) – The mode to use for building the default zero reflection.
- `name` (`str`) – The name of the circuit.

Attributes

ancillas

A list of `AncillaQubit`s in the order that they were added. You should not mutate this.

calibrations

Return calibration dictionary.

The custom pulse definition of a given gate is of the form `{gate.name: {qubits, params}, schedule}`

Deprecated since version 1.3

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.calibrations` is deprecated as of Qiskit 1.3. It will be removed in Qiskit 2.0. The entire Qiskit Pulse package is being deprecated and will be moved to the Qiskit Dynamics repository: <https://github.com/qiskit-community/qiskit-dynamics>. Note that once removed, `qiskit.circuit.quantumcircuit.QuantumCircuit.calibrations` will have no alternative in Qiskit.

cbits

A list of `Cbit`s in the order that they were added. You should not mutate this.

data

The circuit data (instructions and context).

Returns

a list-like object containing the `CircuitInstruction`s for each instruction.

Return type

QuantumCircuitData

duration

The total duration of the circuit, set by a scheduling transpiler pass. Its unit is specified by `Unit`

Deprecated since version 1.3.0

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.duration` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

global_phase

The global phase of the current circuit scope in radians.

instances

Default value: `100`

layout

Return any associated layout information about the circuit

This attribute contains an optional `TranspileLayout` object. This is typically set on the output from `transpile()` by `PassManager.run()` to retain information about the permutations caused on the input circuit by transpilation.

On this page

GroverOperator

Attributes

ancillas

calibrations

cbits

data

duration

global_phase

instances

layout

metadata

num_ancillas

num_captured_vars

num_cbits

num_declared_vars

num_input_vars

num_parameters

num_qubits

num_vars

op_start_times

oracle

parameters

profile

qubits

reflection_qubits

state_preparation

unit

zero_reflection

name

cregs

Was this page helpful?

Yes No

Report a bug or request content on GitHub

z_feature_map	
ZFeatureMap	
ZGate	
zz_feature_map	
ZZFeatureMap	
qiskit.circuit.singleton	
Quantum information (qiskit.quantum_info)	▼
Transpilation	▼
Primitives and providers	▼
Results and visualizations	▼
Serialization (OpenQASM and QPY)	▼
Pulse-level programming	▼
Other	▼
Release notes	▼

There are two types of permutations caused by the `transpile()` function, an initial layout which permutes the qubits based on the selected physical qubits on the `target` and a final layout which is an output permutation caused by `SwapGate` inserted during routing.

metadata

Arbitrary user-defined metadata for the circuit.

Qiskit will not examine the content of this mapping, but it will pass it through the transpiler and reattach it to the output, so you can track your own metadata.

num_ancillas

Return the number of ancilla qubits.

num_captured_vars

The number of real-time classical variables in the circuit marked as captured from an enclosing scope.

This is the length of the `iter_captured_vars()` iterable. If this is non-zero, `num_input_vars` must be zero.

num_cbits

Return number of classical bits.

num_declared_vars

The number of real-time classical variables in the circuit that are declared by this circuit scope, excluding inputs or captures.

This is the length of the `iter_declared_vars()` iterable.

num_input_vars

The number of real-time classical variables in the circuit marked as circuit inputs.

This is the length of the `iter_input_vars()` iterable. If this is non-zero, `num_captured_vars` must be zero.

num_parameters

The number of parameter objects in the circuit.

num_qubits

Return number of qubits.

num_vars

The number of real-time classical variables in the circuit.

This is the length of the `iter_vars()` iterable.

op_start_times

Return a list of operation start times.

This attribute is enabled once one of scheduling analysis passes runs on the quantum circuit.

Returns

List of integers representing instruction start times. The index corresponds to the index of instruction in `QuantumCircuit.data`

Raises

AttributeError *n* – When circuit is not scheduled.

oracle

The oracle implementing a reflection about the bad state.

parameters

The parameters defined in the circuit.

This attribute returns the `Parameter` objects in the circuit sorted alphabetically. Note that parameters instantiated with a `ParameterVector` are still sorted numerically.

Examples

The snippet below shows that insertion order of parameters does not matter.

```
1 >>> from qiskit.circuit import QuantumCircuit, Parameter
2 >>> a, b, elephant = Parameter("a"), Parameter("b"), Parameter("elephant")
3 >>> circuit = QuantumCircuit(1)
4 >>> circuit.rx(b, 0)
5 >>> circuit.rz(elephant, 0)
6 >>> circuit.ry(a, 0)
7 >>> circuit.parameters # sorted alphabetically!
8 ParameterView([Parameter(a), Parameter(b), Parameter(elephant)])
```

Bear in mind that alphabetical sorting might be unintuitive when it comes to numbers. The literal "10" comes before "2" in strict alphabetical sorting.

```
1 >>> from qiskit.circuit import QuantumCircuit, Parameter
2 >>> angles = [Parameter("angle_1"), Parameter("angle_2"), Parameter("angle_10")]
3 >>> circuit = QuantumCircuit(1)
4 >>> circuit.u(*angles, 0)
5 >>> circuit.draw()
6
7 q: ┌ U(angle_1,angle_2,angle_10) ─┐
8
9 >>> circuit.parameters
10 ParameterView([Parameter(angle_1), Parameter(angle_10), Parameter(angle_2)])
```

To respect numerical sorting, a `ParameterVector` can be used.

```
1 >>> from qiskit.circuit import QuantumCircuit, Parameter, ParameterVector
2 >>> x = ParameterVector("x", 12)
3 >>> circuit = QuantumCircuit(1)
4 >>> for x_i in x:
5 ...     circuit.rx(x_i, 0)
6 >>> circuit.parameters
7 ParameterView([
8     ParameterVectorElement(x[0]), ParameterVectorElement(x[1]),
9     ParameterVectorElement(x[2]), ParameterVectorElement(x[3]),
10     ..., ParameterVectorElement(x[11])
11 ])
```

Returns

The sorted `Parameter` objects in the circuit.

prefix

Default value: `"circuit"`

qubits

A list of `Qubit`s in the order that they were added. You should not mutate this.

reflection_qubits

Reflection qubits, on which S0 is applied (if S0 is not user-specified).

state_preparation

The subcircuit implementing the A operator or Hadamards.

unit

The unit that `duration` is specified in.

⚠️ Deprecated since version 1.3.0

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.unit` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

zero_reflection

The subcircuit implementing the reflection about 0.

name

Type: `str`

A human-readable name for the circuit.

qregs

Type: `List[QuantumRegister]`

A list of the `QuantumRegister`s in this circuit. You should not mutate this.

cregs

Type: `List[ClassicalRegister]`

A list of the `ClassicalRegister`s in this circuit. You should not mutate this.